

Standard: Documentation Naming and Structure

[Overview](#)

[Scope](#)

[Repository-Level Documentation](#)

[Repository Core Rule](#)

[README.md - Repository Entry Point](#)

[ARCHITECTURE.md - Repository Architecture Document](#)

[Repository Documentation Folder](#)

[WORKFLOW.md - Repository Workflow Documentation](#)

[Local Documentation](#)

[Local Core Rule](#)

[Local Boundary Definition](#)

[README.md - Local Entry Point](#)

[Local Documentation Folder](#)

[WORKFLOW.md - Local Workflow Documentation](#)

[Canonical Optional Artifacts](#)

[Architecture Decision Records](#)

[Naming Rules](#)

[Workflow Naming](#)

[Ownership and Review](#)

[Maintenance](#)

[Placement Rules](#)

[Must](#)

[Should](#)

[Must not](#)

[Example Structures](#)

[Example A — repository with local feature documentation](#)

[Example B — backend service repository](#)

[Example C — mobile application repository](#)

Overview

This standard defines the enterprise structure, naming, placement, and maintenance rules for repository-level and local documentation.

The goal is to keep documentation predictable, concise, close to the implementation it describes, and usable by both humans and AI-enabled workflows.

Documentation should improve implementation, testing, maintenance, handoff, and cross-team clarity without generating excessive or uncontrolled volume.

This standard defines two documentation levels:

- repository level
- local feature, capability, or service level

The repository level provides shared meaning for the whole codebase.

The local level provides a bounded context for meaningful delivery areas inside the repository.

Both levels should follow the same principles:

- one clear entry point per scope
- stable file names
- explicit purpose
- minimal duplication
- deeper documentation only where justified
- stable section structures where practical
- predictable linkage between prose documentation and formal interface artifacts

Scope

This standard applies to all repositories used in software delivery, whether they contain a frontend application, backend service, mobile application, integration layer, shared library, platform component, or any other bounded software delivery unit.

It also applies to meaningful local boundaries within those repositories. The standard does not assume that all repositories share the same internal structure. Documentation should follow meaningful ownership and delivery boundaries rather than arbitrary folder structures.

Repository-Level Documentation

Repository Core Rule

Each repository must provide a small mandatory documentation core.

The standard repository structure is:

```
<repo>/  
├── README.md  
├── ARCHITECTURE.md  
└── docs/  
    ├── WORKFLOW.md  
    └── optional repository-level documentation
```

The intent is simple:

- `README.md` is mandatory
- `ARCHITECTURE.md` is mandatory
- `docs/` is optional
- `docs/WORKFLOW.md` is optional
- Deeper repository-level documents should be added only when needed

This creates a stable and predictable documentation model without forcing unnecessary files into simple repositories.

README.md - Repository Entry Point

Each repository must provide a `README.md`.

The repository `README.md` is the primary entry point for both humans and AI-enabled workflows.

It should answer only the essential questions:

- What the repository is
- What it owns
- Its main modules, services, or bounded areas
- How to work in the repository
- Where deeper documentation exists

It should remain concise and navigational.

It should not become a general knowledge base.

Where practical, repository `README.md` files should use stable headings such as:

- Purpose
- Scope or Ownership
- Structure
- How to Work Here
- Related Documentation

Equivalent headings may be used where needed, but section structure should remain stable enough to support fast navigation, comparison, review, and AI-assisted loading.

ARCHITECTURE.md - Repository Architecture Document

Each repository must provide an `ARCHITECTURE.md`.

`ARCHITECTURE.md` is the canonical repository-level architecture document.

It should describe the repository-level structure and boundaries, including where relevant:

- major components
- ownership boundaries
- integrations
- important data or control flows
- technical constraints
- security, platform, or runtime considerations
- links to formal interface contracts where those exist

`ARCHITECTURE.md` should define the shared repository's meaning.

It should not be replaced by scattered architecture notes across multiple unrelated files.

Repository Documentation Folder

A repository-level `docs/` folder may be created when a deeper shared context is needed.

The purpose of the repository `docs/` folder is to hold bound repository-wide documentation that cannot be kept concise inside `README.md` or `ARCHITECTURE.md`.

Typical examples include:

- repository-wide workflows
- integration documentation
- testing strategy
- technical constraints
- deployment or release notes
- operational procedures
- architectural decision records (ADR)
- interface contract references

The repository `docs/` folder should remain bounded and purposeful.

It must not become an uncontrolled knowledge base.

WORKFLOW.md - Repository Workflow Documentation

When a repository has meaningful repository-wide workflows, the `docs/` folder should contain a `WORKFLOW.md`.

`docs/WORKFLOW.md` is the stable repository-level workflow entry point.

It may serve one of two roles:

- the workflow itself, when only one repository-wide workflow exists
- a concise workflow overview or index, when multiple repository-wide workflows exist

`WORKFLOW.md` should stay concise. When multiple detailed workflows exist, it should primarily act as the entry point that links to them rather than duplicating them.

Where multiple repository-wide workflows are needed, they should be placed under:

```
docs/workflows/
```

Example:

```
<repo>/  
├── README.md  
├── ARCHITECTURE.md  
└── docs/  
    ├── WORKFLOW.md  
    └── workflows/  
        ├── workflow-functional-<name>.md  
        └── workflow-technical-<name>.md
```

This preserves a stable entry point while allowing multiple workflow files to coexist.

Local Documentation

Local Core Rule

A local documentation boundary should exist only where additional context materially improves implementation, testing, maintenance, or handoff quality.

Each meaningful local boundary should use the following documentation pattern:

```
<feature-or-service>/  
├── README.md  
└── docs/  
    ├── WORKFLOW.md  
    └── optional local documentation
```

The intent is simple:

- `README.md` is the mandatory local entry point
- `docs/` is optional
- `WORKFLOW.md` is optional
- deeper local documents should be added only when needed

This pattern should be applied selectively. It should not be applied to every folder in the repository.

Local Boundary Definition

A local documentation boundary is justified when one or more of the following are true:

- the area has non-obvious business behavior
- the area has non-trivial technical constraints
- the area interacts with external systems
- the area has a distinct delivery or validation workflow
- the area is frequently modified by multiple teams
- the area is a recurring source of ambiguity or defects
- the area exposes interfaces or contracts that require stable local reference

Where these conditions do not exist, additional local documentation should not be created.

README.md - Local Entry Point

Each meaningful local boundary must contain a `README.md`.

The local `README.md` is the first local context entry point for both humans and AI-enabled workflows.

It should remain concise and answer only the essential questions:

- what this area does
- what it owns
- what its boundaries are
- which dependencies or external systems it relies on
- which contracts or interfaces it exposes or consumes, where relevant
- where deeper local documentation exists, if any

The local `README.md` should not duplicate repository-level documentation unless a short local restatement is required for clarity.

Where practical, local `README.md` files should use stable headings such as:

- Purpose
- Responsibilities
- Boundaries
- Dependencies or External Systems
- Related Documentation

Local Documentation Folder

A local `docs/` folder may be created only when the local context cannot be kept clear and concise within the `README.md`.

The purpose of the local `docs/` folder is to hold deeper, bounded documentation specific to that local area.

Typical examples include:

- workflow documentation
- functional requirement details
- external interaction notes
- testing rules
- technical constraints
- recovery or operational procedures
- local architectural decisions
- contract notes and references

The local `docs/` folder must remain scoped to the local boundary.

It must not become a parallel repository-wide documentation structure.

WORKFLOW.md - Local Workflow Documentation

When a local feature, capability, or service has a meaningful bounded workflow, the local documentation folder should contain a `WORKFLOW.md`.

`WORKFLOW.md` is the stable local workflow entry point.

It may serve one of two roles:

- the workflow itself, when only one local workflow exists
- a concise workflow overview or index, when multiple local workflows exist

`WORKFLOW.md` should stay concise. When multiple detailed workflows exist, it should primarily act as the entry point that links to them rather than duplicating them.

Where multiple local workflows are needed, they should be placed under:

```
docs/workflows/
```

Example:

```
<feature-or-service>/  
├── README.md  
└── docs/  
    ├── WORKFLOW.md  
    └── workflows/  
        ├── workflow-functional-<name>.md  
        └── workflow-technical-<name>.md
```

This preserves a stable local entry point while allowing multiple workflow files to coexist.

Canonical Optional Artifacts

The following artifacts are optional, but when they are needed they should use predictable names and locations.

Recommended optional documents include:

- `testing.md`
- `technical-constraints.md`
- `deployment.md`
- `release.md`
- `integration-<name>.md`
- `functional-requirement-<name>.md`
- `operational-procedure-<name>.md`

These documents may exist at repository level or local level depending on scope.

They should be added only where they materially improve understanding, implementation, validation, operations, or handoff quality.

Architecture Decision Records

Where architectural decisions require traceability, repositories or local boundaries may maintain architecture decision records.

The recommended location is:

```
docs/adr/
```

Recommended naming is:

- `adr-001-<name>.md`
- `adr-002-<name>.md`

ADRs should be used selectively for decisions that materially affect architecture, integration, platform behavior, operational behavior, or long-term maintainability.

ADRs should not replace `ARCHITECTURE.md`.

`ARCHITECTURE.md` remains the canonical repository-level architecture overview, while ADRs capture significant decision history.

Naming Rules

Workflow Naming

Workflows may serve either functional or technical purposes. Both types may exist at repository level and local level.

To avoid ambiguity, workflow names should explicitly identify their type.

Functional workflows should use:

```
workflow-functional-<name>.md
```

Examples:

- `workflow-functional-checkout-handoff.md`
- `workflow-functional-ux-defect-review.md`

Technical workflows should use:

```
workflow-technical-<name>.md
```

Examples:

- `workflow-technical-state-recovery.md`
- `workflow-technical-regression-validation.md`

This distinction should be applied consistently at both repository and local levels.

Ownership and Review

Documentation quality must have clear ownership.

Repository-level documentation should be owned by the repository owner or owning team.

Local documentation should be owned by the team responsible for the local feature, capability, or service boundary.

Documentation changes **should be reviewed as part of normal change** review whenever behavior, interfaces, workflows, dependencies, constraints, or operating rules materially change.

Ownership must be clear enough that stale, duplicated, or misleading documentation can be corrected quickly.

Maintenance

Documentation should be **updated whenever behavior, interfaces, workflows, dependencies, constraints, contracts, or operating rules materially change.**

Documentation is strongest when it evolves with the implementation it describes.

Placement Rules

The following rules apply.

Must

- Every repository must have a `README.md`.
- Every repository must have an `ARCHITECTURE.md`.
- Every meaningful local boundary must have a `README.md`.
- `WORKFLOW.md` must be used as the stable workflow entry point when workflow documentation exists.
- Functional and technical workflows must be explicitly distinguished by file name when that distinction is relevant.
- Repository-level documentation must stay scoped to repository-wide meaning.
- Local documentation must stay scoped to the feature, capability, or service it describes.
- Formal interface contracts must use a predictable canonical location when they exist.
- Prose documentation must reference formal contracts where those contracts are part of the delivery boundary.

Should

- Documentation should stay close to the implementation it describes.
- `README.md` files should remain short and navigational.
- `WORKFLOW.md` should remain concise and act as an index when multiple workflows exist.
- Deeper documents should exist only where they materially improve understanding or delivery quality.
- Local documents should link to repository-wide canonical standards rather than restating them.

- Repository-level `docs/` folders should remain bounded and purposeful.
- Entry-point documents should use stable headings where practical.
- Architectural decisions with long-term impact should be captured in ADRs where justified.

Must not

- Documentation must not be created for every folder by default.
- Local documentation must not duplicate repository-wide standards unnecessarily.
- Repository-level or local `docs/` folders must not become uncontrolled mini knowledge bases.
- Workflow names must not omit whether they are functional or technical when that distinction matters.
- Architecture meaning must not be scattered across undocumented or inconsistently named files.
- Formal contracts must not be replaced by prose-only descriptions when a machine-readable source of truth is required.

Example Structures

Example A — repository with local feature documentation

```
frontend-app/  
├── README.md  
├── ARCHITECTURE.md  
└── docs/  
    ├── WORKFLOW.md  
    ├── contracts/  
    ├── adr/  
    └── workflows/  
        ├── workflow-functional-release-handoff.md  
        └── workflow-technical-deployment-validation.md  
  
src/routes/login/  
├── README.md  
└── docs/  
    ├── WORKFLOW.md  
    ├── functional-requirement-login.md  
    ├── integration-IAM.md  
    └── contracts/
```

```
└─ workflows/
   └─ workflow-functional-checkout-handoff.md
   └─ workflow-technical-state-recovery.md
```

Example B — backend service repository

```
payments-service/
└─ README.md
└─ ARCHITECTURE.md
└─ docs/
   └─ WORKFLOW.md
   └─ integration-payment-gateway.md
   └─ technical-constraints.md
   └─ adr/
      └─ workflows/
         └─ workflow-technical-release-validation.md

services/reconciliation/
└─ README.md
└─ docs/
   └─ WORKFLOW.md
   └─ technical-constraints.md
   └─ contracts/
      └─ workflows/
         └─ workflow-technical-reconciliation-job.md
```

Example C — mobile application repository

```
mobile-app/
└─ README.md
└─ ARCHITECTURE.md
└─ docs/
   └─ WORKFLOW.md
   └─ adr/
```

- └─ workflows/
 - └─ workflow-functional-mobile-release-handoff.md
 - └─ workflow-technical-device-validation.md

app/onboarding/

- └─ README.md
- └─ docs/
 - └─ WORKFLOW.md
 - └─ functional-requirement-mobile-onboarding.md
 - └─ workflows/
 - └─ workflow-functional-onboarding-handoff.md
 - └─ workflow-technical-device-permission-validation.md